HAW HAMBURG

EXPOSÉ
Leonhard Lüdeke

Entwicklung einer Referenzarchitektur eines Wissen orientierten Multi-Agenten-Systems unter Berücksichtigung einer Threat-Modelling Analyse

FAKULTÄT INFORMATIK UND DIGITALE GESELLSCHAFT

Faculty of Computer Science and Digital Society

Betreuung durch: Prof. Dr. Thomas Lehmann
Eingereicht am: XX, 2026

HOCHSCHULE FÜR ANGEWANDTE WISSENSCHAFTEN HAMBURG
Hamburg University of Applied Sciences

Inhaltsverzeichnis

1 Kontext und Zielsetzung **1**
1.1 Arbeitstitel . . . 1
1.2 Ausgangslage und Motivation . . . 1
1.3 Problemstellung . . . 2
1.4 Zielsetzung der Arbeit . . . 3
1.5 Forschungsfragen . . . 4

2 Grundlagen und Bewertungsrahmen **5**
2.1 Theoretische und fachliche Grundlagen . . . 5
2.2 Agentische KI und Multi-Agenten-Systeme . . . 6
2.3 Agentenkommunikation und Interoperabilität . . . 7
2.4 Threat Modeling für LLM-basierte Agentensysteme . . . 8

3 Architektur **9**
3.1 Anforderungen für Architektur . . . 9
3.2 Bisherige Überlegungen zur Architektur . . . 9

4 Methodik und Struktur **11**
4.1 Methodisches Vorgehen . . . 11
4.2 Vorläufige Kapitelstruktur . . . 12

5 Beitrag, Risiken und Arbeitsplan **12**
5.1 Erwarteter Beitrag . . . 12
5.2 Annahmen, Unsicherheiten und offene Punkte . . . 13

6 Anhang **13**

Literatur **16**

Keywords: KI-Agenten, Multi-Agent-Systeme, Threats

1 Kontext und Zielsetzung

1.1 Arbeitstitel

Entwicklung einer Referenzarchitektur eines wissensorientierten Multi-Agenten-Systems unter Berücksichtigung einer Threat-Modelling Analyse

1.2 Ausgangslage und Motivation

Große Sprachmodelle werden im aktuellen Diskurs zunehmend nicht mehr nur als Grundlage einfacher dialogorientierter Anwendungen verstanden, sondern als Bausteine agentischer Systeme, in denen mehrere spezialisierte Komponenten Aufgaben übernehmen, Informationen austauschen und arbeitsteilig zur Bearbeitung komplexerer Probleme beitragen. [29, 14, 8]

Diese Entwicklung lässt sich als Fortführung klassischer Konzepte aus der Multi-Agenten-Forschung und verteilten Systemen interpretieren, die durch LLM-basierte Sprachverarbeitung, flexible Werkzeugnutzung und dynamische Kontextverarbeitung [1] erweitert werden. [14, 32, 27] Mit dieser funktionalen Erweiterung steigt zugleich die technische und organisatorische Komplexität. [14, 28, 12]

Während klassische Multi-Agenten-Systeme bereits Fragen der Koordination, Kooperation und Kommunikation adressieren, entstehen bei LLM-basierten Architekturen zusätzliche Unsicherheiten durch auf Wahrscheinlichkeits-basierten Modellentscheidungen. [28, 12, 21] Hierbei stellt zum Beispiel die Entscheidung, welcher Agent mit welchem Input angesprochen wird, eine Schwierigkeiten da. Weitere Herausforderung sind die Einbindung externer Tools und die Verarbeitung heterogener Kontextquellen.

Für Unternehmen ist daher nicht nur relevant, ob ein Multi-Agenten-System grundsätzlich leistungsfähig erscheint, sondern ob es unter realen Randbedingungen nachvollziehbar, beherrschbar und sicher betrieben werden kann. [21, 12]

[1] Kontext verändert sich durch zusätzliche Nutzereingaben (Human in the Loop), Suchergebnisse und durch LLM selbst erstellte Zwischenergebnisse.

Damit gewinnt die Fragestellung, wie LLM basierte Multi-Agenten-Systeme in bestehende technische und organisatorische Strukturen eingebettet werden können, immer mehr an Relevanz. Somit rücken Architekturentscheidungen in den Mittelpunkt, die nicht allein funktional, sondern zugleich unter Sicherheitspunkten begründet werden müssen. [28, 17] Zusätzlich sollten Governance-Aspekte mit in die Überlegungen einfließen und können als Basis für eine Risikoanalyse dienen.

1.3 Problemstellung

In aktuellen Diskussionen zu agentischen Systemen stehen häufig der Funktionsumfang, Automatisierungspotenziale und die Qualität generierter Ergebnisse im Vordergrund. [17, 12, 33]

Weniger systematisch behandelt wird dagegen die Frage, wie ein solches System im Unternehmenskontext so entworfen werden kann, dass Sicherheitsaspekte nicht erst nachträglich ergänzt, sondern bereits auf Architekturebene berücksichtigt werden. [25, 28, 12]

Gerade bei LLM-basierten Multi-Agenten-Systemen erweitert sich die Angriffsfläche gegenüber einfacheren Einzelagenten- oder Chatbot-Ansätzen deutlich. [28, 12, 6]

Risiken ergeben sich nicht nur aus direkten Benutzereingaben, sondern auch aus Agentenkommunikation, Delegationsmechanismen, Tool-Aufrufen, Retrieval-Prozessen, Protokollschichten und der Weitergabe von Kontexten oder Zwischenergebnissen. [28, 17, 12, 7, 33] Daraus folgt, dass eine rein funktionale Betrachtung des Systementwurfs für die vorliegende Problemstellung nicht ausreicht. [28, 25]

Die zentrale Problemstellung der Arbeit besteht daher darin, eine nachvollziehbare Verbindung zwischen funktionaler Architekturentwicklung und systematischem Threat Modeling herzustellen. Untersucht wird, wie sich ein LLM-basiertes Multi-Agenten-System so konzipieren lässt, dass sowohl fachliche Anforderungen als auch Schutzbedarfe, Vertrauensgrenzen und Gegenmaßnahmen konsistent in einer Zielarchitektur abgebildet werden können. [17, 12]

1.4 Zielsetzung der Arbeit

Ziel der Arbeit ist die Konzeption eines LLM-basierten Multi-Agenten-Systems im Unternehmenskontext sowie die sicherheitsorientierte Bewertung dieses Entwurfs mithilfe einer strukturierten Threat-Modeling-Analyse. Im Vordergrund steht damit nicht primär die vollständige produktive Implementierung, sondern die Entwicklung einer fachlich plausiblen, technisch realisierbaren und sicherheitsorientiert begründeten Referenzarchitektur.

Das Multi-Agenten-System hat zum Ziel, bei Recherche-, Analyse- und Syntheseaufgaben zu assistieren. Hierfür werden LLM-basierte Agenten Informationen aus heterogenen internen und externen Quellen beschaffen, analysieren, zusammenführen und bewerten, um ein nachvollziehbares Arbeitsergebnis zur menschlichen Prüfung oder Entscheidung vorzubereiten.

Die Arbeit verfolgt dabei drei miteinander verbundene Teilziele. Erstens soll eine Referenzarchitektur entwickelt werden, in der Rollen, Komponenten, Schnittstellen und Kommunikationsbeziehungen der beteiligten Entitäten klar beschrieben sind. Zweitens sollen relevante Bedrohungen systematisch identifiziert und entlang von Komponenten, Datenflüssen und Vertrauensgrenzen analysiert werden. Drittens sollen daraus konkrete Maßnahmen abgeleitet werden, die zeigen, wie Sicherheitsanforderungen in ein konsistentes Systemdesign überführt werden können. [19, 28, 17]

Die daraus entstehende Referenzarchitektur soll als Entscheidungsgrundlage für spätere produktive Umsetzung dienen. Die Entwurfsentscheidungen werden in einem Prototypen realisiert und die Funktionalität anhand von Anwendungsfällen untersucht.

1.5 Forschungsfragen

Die übergeordnete Forschungsfrage lautet:

Wie kann eine LLM-basierte Multi-Agenten-Referenzarchitektur für wissensintensive Unternehmensanwendungen, so entwickelt und evaluiert werden, dass funktionale, organisatorische und sicherheitsbezogene Anforderungen nachvollziehbar auf Architekturentscheidungen, Bedrohungen und Gegenmaßnahmen miteinander verknüpft sind?

Zur Beantwortung dieser Leitfrage werden folgende untergeordnete Forschungsfragen betrachtet:

- Welche Merkmale, Akteure, Aufgaben, Informationsquellen und Interaktionsformen kennzeichnen wissensintensive Unternehmensanwendungen?
- Welche funktionalen, organisatorischen, technischen und sicherheitsbezogenen Anforderungen ergeben sich aus den betrachteten Unternehmensanwendungen und den Perspektiven der relevanten Stakeholder?
- Welche klassischen sowie LLM- und Multi-Agenten-spezifischen Bedrohungen entstehen durch Benutzerinteraktion, Agentenkommunikation, Delegation, Tool-Nutzung und Modellanbindung?
- Mit welchen architektonischen und organisatorischen Maßnahmen lassen sich diese Bedrohungen angemessen mindern?
- In welchen Punkten erzeugt der Multi-Agenten-Ansatz einen Mehrwert gegenüber einfacheren Single-Agent-Referenzarchitektur, und welche Sicherheitsrisiken ergeben sich dadurch?

2 Grundlagen und Bewertungsrahmen

2.1 Theoretische und fachliche Grundlagen

Die theoretische Grundlage der Arbeit verbindet klassische Konzepte der Multi-Agenten-Forschung mit neueren Ansätzen LLM-basierter agentischer Systeme. [29, 14, 22] Ausgangspunkt ist der in der Literatur etablierte Agentenbegriff, nach dem ein Agent als in einer Umgebung situierte Softwareeinheit verstanden werden kann, die flexibel und autonom im Sinne ihrer Entwurfsziele handelt. [29, 30, 22]

Auf dieser Basis lässt sich ein Multi-Agenten-System als eine Architektur beschreiben, in der mehrere autonome oder teilautonome Einheiten kooperieren, kommunizieren und ihre Aktivitäten koordinieren, um individuelle oder gemeinsame Ziele zu verfolgen. [14, 18, 22, 8]

Damit sind heutige LLM-basierte Agentensysteme als Erweiterung klassischer Agenten- und MAS-Konzepte zu verstehen, bei denen Sprachmodelle neue Fähigkeiten der Interaktion, Werkzeugnutzung und Kontextverarbeitung bereitstellen, ohne die grundlegenden Fragen nach Autonomie, Koordination und Verantwortungsverteilung obsolet zu machen. [8, 32, 14, 22]

Diese Veränderung der Agentensysteme hin zu LLM-basierten Agentensystemen lässt neue Angriffsvektoren entstehen [25, 17], die neben den klassischen Bedrohungen für Multi-Agenten-Systemen bestehen. [28]

Um diese Bedrohungen zu Analysieren werden Vertrauensgrenzen und Datenflüsse modelliert. Für die Gestaltung von Gegenmaßnahmen bedarf es einer Berücksichtigung von kontrollierten Schnittstellen, das Prinzip minimaler Rechte, sowie Nachvollziehbarkeit und Beherrschbarkeit. [28, 25]

2.2 Agentische KI und Multi-Agenten-Systeme

Agentische KI bezeichnet Systeme, die nicht nur einzelne Antworten erzeugen, sondern zielgerichtet Aufgaben bearbeiten, Zwischenschritte planen, externe Werkzeuge verwenden und Teile ihres Handlungsablaufs selbst steuern. [29, 15]

Von einem Multi-Agenten-System wird gesprochen, wenn mehrere spezialisierte Agenten miteinander interagieren und durch Kommunikation, Delegation oder Orchestrierung zu einem Gesamtergebnis beitragen.[14, 30, 22]

Für die Arbeit ist es wichtig, zwischen dem traditionellen Agentenbegriff und heutigen LLM-basierten Agenten zu unterscheiden. [22, 8]

Die Grundlage, welche Aspekte bei der praktischen Entwicklung von Agenten und Multi-Agenten-Systeme betrachtet werden müssen, hat sich nicht maßgeblich verändert. Das zeigen auch die 3 relevantesten Teilaspekte [22, 14] die moderne Frameworks wie LangGraph, AutoGen, ADK, CrewAI und viele weitere adressieren:

- Begründung (Reasoning): Wie wird das Handeln eines Agenten argumentiert?
- Speicher (Memory): Entscheidungen müssen in einen Kontext gesetzt werden können und bauen sich auf (episodischer Aufbau).
- Nutzung von Tools (Tool use): Ansprechen von externen Systemen (Server, APIs, Browser) mittels strukturierter Eingabeaufforderungen.

Diese Kernelemente von Multi-Agenten-Systeme hat Sycara Jahre 1996 mit der Entwicklung eines Multi-Agenten Framework names 'Retsina' adressiert. Darin wurde eine wiederverwendbare Multi-Agenten-Infrastruktur aufgebaut die Planung, Informationsbeschaffung und Kooperation realisierten. [24]

Jedoch beruhen Klassische Agentensysteme typischerweise stärker auf expliziten Zustandsmodellen, Regeln und formalen Interaktionsmustern, während LLM-basierte Agenten in hohem Maß durch probabilistische Generierung geprägt sind. [29, 14, 8] Dabei setzen sich die einzelnen Agenten eigenständig Ziele und Interpretieren diese in natürlicher Sprache. Daraus entstehen neue Freiheitsgrade, aber auch neue Unsicherheiten in Bezug auf Robustheit, Fehlverhalten und Interpretierbarkeit. [27, 21]

Ein Multi-Agenten-Ansatz ist deshalb nicht per se vorteilhaft. [14, 32, 16] Sein Einsatz muss über konkrete Anforderungen wie Spezialisierung, Modularisierung, Trennung von

Verantwortlichkeiten oder kontrollierte Delegation begründet werden. [14, 29, 31] Andernfalls droht zusätzliche Komplexität ohne klaren funktionalen oder sicherheitlichen Mehrwert. [28, 32, 16, 21]

2.3 Agentenkommunikation und Interoperabilität

Kommunikation ist ein zentrales Strukturmerkmal von Multi-Agenten-Systemen. [22, 9, 14] Bereits in der klassischen Literatur wird deutlich, dass die Leistungsfähigkeit verteilter Agentensysteme wesentlich davon abhängt, wie Informationen zwischen beteiligten Einheiten ausgetauscht, interpretiert und in koordiniertes Handeln überführt werden. [14, 22, 8]

Historische Arbeiten zu KQML und spätere Standardisierungsbestrebungen, etwa im Umfeld von FIPA (Foundation for Intelligent Physical Agents), verdeutlichen, dass Agentenkommunikation nicht nur als technischer Nachrichtentransport, sondern als strukturierte Interaktion mit syntaktischen, semantischen und pragmatischen Ebenen verstanden werden sollte. [9, 5, 8]

Für LLM-basierte Multi-Agenten-Systeme bleibt diese Einsicht zentral. [27, 32] Zwar erfolgen Kommunikationsprozesse heute häufig über natürlichsprachliche oder halb strukturierte Kontexte, welches eine gewisse Flexibilität bietet, jedoch dies die Gefahr von Missverständnissen, Kontextverlusten und unkontrollierten Seiteneffekten. [32, 12]

Interoperabilität bedeutet deshalb in der vorliegenden Arbeit nicht nur technische Anschlussfähigkeit, sondern eine verlässlich kontrollierbare Zusammenarbeit trotz unterschiedlicher Rollen, Zustände und Systemgrenzen. [5, 9, 6]

2.4 Threat Modeling für LLM-basierte Agentensysteme

Threat Modeling bezeichnet wiederholbare systematische Verfahren zur Identifikation, Strukturierung und Bewertung potenzieller Bedrohungen innerhalb eines Systems. [19, 23] Dabei umfasst das Threat Modeling diverse Framworks und Methoden wie STRIDE, PASTA, LINDDUN, 'attack trees', 'abuse cases' und viele mehr. [19]

Auch für LLM-basierte Multi-Agenten-Systeme sind diese Ansätze geeignet, da Risiken nicht nur aus einzelnen Komponenten, sondern aus Interaktionsketten zwischen Benutzenden, Agenten, Protokollen, Tools, Datenquellen und externen Diensten entstehen. [17]

Im Unterschied zu einer rein Komponenten bezogenen Sicherheitsbetrachtung erlauben Threat Modeling Framwork's eine methodische Analyse entlang konkreter Datenflüsse, Rollen und Vertrauensgrenzen. [23, 19, 28]

Dadurch können sowohl klassische Bedrohungen verteilter Agentensysteme als auch neuere LLM-spezifische Risiken erfasst werden. [17, 3] Dazu zählen etwa Prompt Injection, manipulierte Kontextquellen und Protokollschwächen. Ebenfalls stellen Fehlerhafte Tool-Aufrufe, welche durch reines Routing auf Basis von LLM-Entscheidungen getroffen werden, entstehen können ein Risiko da. Zusätzlich bedarf der sichere Zugriff auf getrennte Systeme eine dezidierte Betrachtung, da dies User*innen Potenzielle Zugriff auf sensible Informationen ermöglicht. Dies fällt unter die Thematik von Äccess Controllünd wird in dieser Arbeit nur am Rande betrachtet. [12, 6, 33, 17]

Diese Arbeit wird die 'OWASP Top 10 for LLM Applications' [11] als Orientierung für Risikokategorien verwendet und klassische Threat Modeling Frameworks für die Beantwortung der Forschungsfragen nutzen.

3 Architektur

3.1 Anforderungen für Architektur

Die Referenzarchitektur wird aus fachlichen, organisatorischen, technischen und sicherheitsbezogenen Anforderungen abgeleitet. Diese werden auf Basis von fachlichen Interviews, technischen Randbedingungen und der Literatur gebildet.

Daher ist der Ausgangspunkt nicht die Frage, was für Technologien verfügbar sind sondern welchen Anwendungsfall das System erfüllen soll und unter welchen Randbedingungen es betrieben wird. Hier sind sowohl Unternehmerischen Richtlinien wie auch Technische Begebenheiten zu berücksichtigen, sodass sich die Komponenten des Multi-Agenten-System in einer containerisierten Umgebung einbinden lassen.

Durch die Analytische Betrachtung durch geeignete Threat-Modeling Prozesse werden weitere Designentscheidungen getroffen, welche fortlaufende Architekturentscheidungen begründen sollen.

Dies könnten zum Beispiel zusätzliche Kontrollmechanismen, Protokollentscheidungen oder auch stärkere zentralisierung/verteiltes Design des Multi-Agenten-Systems sein.

3.2 Bisherige Überlegungen zur Architektur

Die bisherige Zielarchitektur orientiert sich an einem modularen Multi-Agenten-Ansatz mit getrennten Verantwortlichkeiten. Ein zentraler Orchestrator und/oder Workflow-Controller erscheint für Multi-Agenten-Systeme als plausible Kernkomponente. [13, 14, 26]

Dieser nimmt Anfragen entgegen, strukturiert Aufgaben, delegiert Teilaufgaben an spezialisierte Agenten und konsolidiert Ergebnisse. [24, 31] Diese Verarbeitungskette wird durch eine Kombination aus deterministischen und LLM-basierten Entscheidungen realisiert. Damit werden Koordination, Kontrolllogik und Fehlerbehandlung in dieser Instanz gebündelt, in welcher der Gesamtprozess transparent strukturiert werden kann. [28]

Insgesamt erhöhen deterministische Abläufe die Kontrollierbarkeit des Systems, jedoch auf Kosten der Flexibilität in Hinblick auf Erweiterung mit zusätzlichen Agenten. Diese müssen nämlich in deterministische Abläufe mit eingebunden werden.

Der im Anhang beschriebene Anwendungsfall 6 beschreibt ein MAS das den Nutzer*innen bei unterschiedlichen Komplexen Wissensaufgaben assistiert. Dem Orchestrator stehen für diese Aufgabenbewältigung aktuell 3 spezialisierte Agenten zur Verfügung.

Ein 'Websearch' Agent, welcher als Input einen String erhält, diesen Aufbereitet und eine oder mehrere Websuchen über die Google API durchführt. Die Ergebnisse werden neu aggregiert und nach einem durch das LLM vergebenen 'relevance score' sortiert. Anschließend werden die Suchergebnisse zurückgegeben.

Zusätzlich steht ein 'Planner' Agent zur Verfügung. Dieser benötigt wiederum einen String als Input, welcher durch ein LLM aufbereitet wird, indem der Plan konkretisiert und durch Zielkriterien [2] ergänzt wird. Dies ist die Basis für eine Ausarbeitung eines Planes. Zurückgegeben wird der Plan als ein in Markdown formatierter String.

Als eine Kontrollinstanz dient der 'Judge' Agent. Dieser bekommt Input stellt wiederum ein String da. In verschiedenen Stationen des Agenten wird geprüft ob der String den Anforderungen welche Nutzer*innen mitgeben oder der Judge Agent selbst aufgestellt hat entspricht. Anschließend wird auf dieser Basis ein Feedback entwickelt und zurückgeliefert.

Es wird angenommen das alle beteiligten Systeme innerhalb des gleichen Subnetzes in einem Kubernetes Cluster [2] und sind mittels LangChain, LangGraph in Python [10, 4] und einen FastAPI [20] Endpunkt als Gateway realisiert. Es ist bisher nicht beabsichtigt, das die Spezialagenten direkt miteinander Kommunizieren. Die komplette Koordination von Aufgaben und Zwischenergebnissen liegt bei dem Orchestrator Agent.

Als User-Interface kann OpenWebUI [1] als Frontend Layer dienen, um das Multi-Agenten-System [3] Anzubinden. Als reines Frontend Layer sind sehr viele Leistungen von OpenWebUI ungeeignet, jedoch sind viele Möglichkeiten wie die Einbindung von diversen LLM's, RAG Aufbau, Authentifizierungsfunktionen und weitere Methoden für Unternehmen von potenziellem Interesse. Somit könnte eine beispielhafte Einbindung eines MAS in OpenWebUI als Entscheidungshilfe für diese Open Source Lösung dienen.

Die bisherigen Überlegungen zur architektonischen Umsetzung, sowie bisher maßgeblich beteiligte Systeme sind im angegebenen Diagramm dargestellt. 1 Dies ist vorerst nur als zusätzliche veranschaulichte Information zu betrachten.

[2] Dies umfasst: Umfang der Ausarbeitung, Erwartete Struktur des Ergebnisses, Schwerpunkt: Theorie, Machbarkeit, Praxis, Beweisart: empirisch, anekdotisch

[3] Wobei hier der Orchestrator als äquivalent zum MAS angeführt werden kann.

4 Methodik und Struktur

4.1 Methodisches Vorgehen

Methodisch kombiniert die Arbeit konzeptionelle Architekturentwicklung mit strukturierter Sicherheitsanalyse.

Zunächst werden auf Basis einschlägiger Literatur sowie der gegebenen Rahmenbedingungen Anforderungen, Randbedingungen und Zielkriterien herausgearbeitet. [13, 15, 14, 22, 24, 27] Hierbei sind neben den Literatur basierten Rahmenbedingungen, die Unternehmerischen Anforderungen von maßgeblicher Bedeutung. Hierfür werden Fachinterviews mit Abteilungsleitern eines IT-Unternehmens geführt. Diese Interviews orientieren sich an den vordefinierten Forschungsfragen in Kombination mit Anwendungsfällen (Anhang 6). Mit diesen Ergebnissen werden Anforderungen, Randbedingungen und Zielkriterien konkretisiert als auch notwendige Sicherheitsanforderungen identifiziert. Hierbei werden nur Branchenspezifische Aspekte und keine expliziten Unternehmensspezifische Informationen verwendet.

Anschließend wird daraus eine Zielarchitektur für ein LLM-basiertes Multi-Agenten-System abgeleitet, in der Rollen, Schnittstellen, Kommunikationsbeziehungen und zentrale Vertrauensgrenzen beschrieben werden.

Im nächsten Schritt wird diese Architektur in Komponenten, Datenflüsse und potenzielle Angriffsflächen zerlegt.

Darauf aufbauend erfolgt das Threat Modeling, das klassische Kategorien verteilter Systeme mit LLM-spezifischen Angriffsmustern verbindet. [28, 25, 17, 3, 11]

Die identifizierten Risiken werden anschließend in technische und organisatorische Gegenmaßnahmen übersetzt und hinsichtlich ihrer Wirksamkeit sowie verbleibender Restrisiken diskutiert. [17, 28, 11, 23, 12, 21]

Auf diese Weise entsteht ein methodischer Ansatz, der Entwurf, Analyse und Bewertung nicht voneinander trennt, sondern systematisch miteinander verknüpft.

4.2 Vorläufige Kapitelstruktur

Die Kapitelstruktur beginnt mit der Darstellung des Problemkontextes über die theoretischen Grundlagen und die Anforderungsanalyse zur Architekturkonzeption, Erstellung einer Sicherheitsanalyse und endet mit einer Evaluation und Beantwortung der Forschungsfragen.

Eine mögliche Struktur umfasst die Kapitel Einleitung, theoretische Grundlagen, methodisches Vorgehen, Anforderungsanalyse, Architekturkonzeption, Threat Modeling, Evaluation, Diskussion und Fazit.

Dabei ist besonders wichtig, dass der Threat-Modeling Teil nicht als isolierter Sicherheitseinschub erscheint, sondern als integraler Bestandteil der Architekturentwicklung nachvollziehbar wird.

5 Beitrag, Risiken und Arbeitsplan

5.1 Erwarteter Beitrag

Der erwartete Beitrag liegt in der systematischen Verbindung von Architekturkonzeption und Threat Modeling für ein LLM-basiertes Multi-Agenten-System im Unternehmenskontext.

Der Mehrwert besteht in einer nachvollziehbaren sicherheitsorientierten Entwurfsperspektive, die Komponenten, Kommunikationsbeziehungen, Vertrauensgrenzen, Risiken und Gegenmaßnahmen gemeinsam betrachtet. Dabei soll aufgezeigt werden, wie ein strukturiertes Threat Modeling Architekturentscheidungen beeinflusst und welche Restrisiken trotz spezifischer Maßnahmen bestehen bleiben.

Durch eine Gegenüberstellung der Entwickelten Multi-Agenten-Referenzarchitektur mit einer einfacheren (Single-Agent) Referenzarchitektur, können Bedingungen herausgearbeitet werden, unter denen Spezialisierung und kontrollierte Delegation den zusätzlichen Aufwand für den Betrieb und Implementierung eines Multi-Agenten-System rechtfertigen.

Wissenschaftlich zeigt sie, wie sich klassische Konzepte der Multi-Agenten-Forschung und etablierte Sicherheitsüberlegungen auf neuere agentische Architekturen mit LLM-Komponenten übertragen und konkretisieren lassen.

5.2 Annahmen, Unsicherheiten und offene Punkte

Die Arbeit basiert auf mehreren Annahmen, die offen benannt werden müssen. Dazu zählen Annahmen über den vorgesehenen Nutzungskontext, über technische Integrationsmöglichkeiten, über interne und externe Kommunikationsbeziehungen sowie über potenzielle Angreifer und Missbrauchsszenarien.

Es ist wichtig, solche Annahmen transparent zu machen, da Bedrohungsbilder stark von Systemgrenzen, Rollenmodellen und Kommunikationsmustern abhängen.

Gleichzeitig bestehen Unsicherheiten, etwa hinsichtlich späterer Implementierungsdetails, konkreter Schnittstellen, Freigabeprozesse oder real verfügbarer Infrastruktur.

Diese Unsicherheiten mindern nicht den Wert der Arbeit, müssen jedoch explizit als Begrenzung des Geltungsbereichs berücksichtigt werden.

6 Anhang

Anwendungsklasse Die Anwendungsfallklasse umfasst komplexe, informations- und dokumentenzentrierte Unternehmensaufgaben, bei denen mehrere spezialisierte LLM-basierte Agenten Informationen aus heterogenen internen und externen Quellen beschaffen, analysieren, zusammenführen und bewerten, um ein nachvollziehbares Arbeitsergebnis zur menschlichen Prüfung oder Entscheidung vorzubereiten.

Mögliche formulierungen für Anwendungsfälle - Assistierte Bearbeitung komplexer wissensintensiver Unternehmensaufgaben - Agentengestützte Recherche, Analyse und Synthese heterogener Informationen - Teilstrukturierte, wissensintensive Unternehmensprozesse mit dynamischer Aufgabenzerlegung

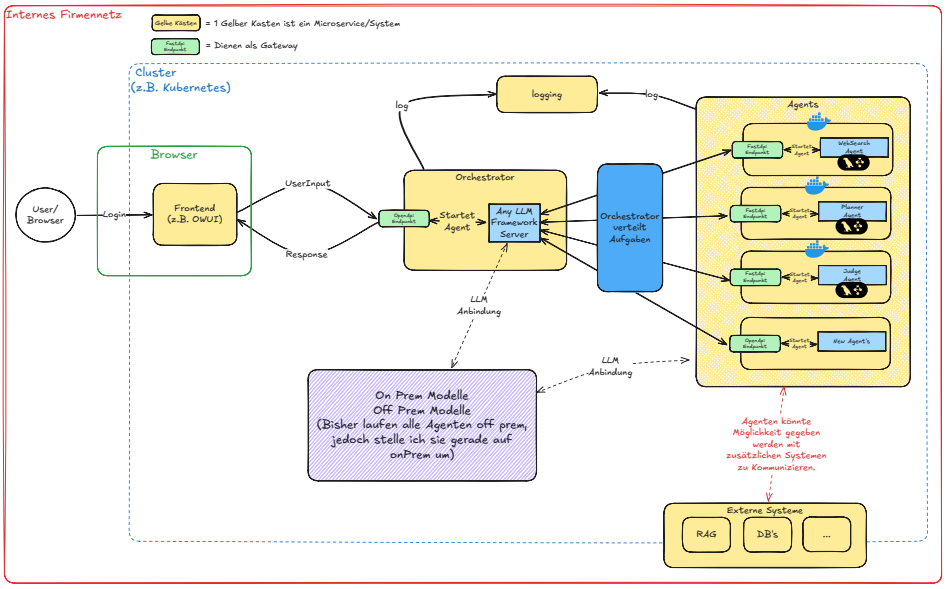

Internes Firmennetz
= 1 Gelber Kasten ist ein Microservice/System
= Dienen als Gateway
Cluster
(z.B. Kubernetes)
logging
log
log
Agents
Browser
User/
Browser
Login
Frontend
(z.B. OWUI)
UserInput
Response
Orchestrator
Startet
Agent
Any LLM
Framework
Server
Orchestrator
verteilt
Aufgaben
LLM
Anbindung
LLM
Anbindung
On Prem Modelle
Off Prem Modelle
(Bisher laufen alle Agenten off prem,
jedoch stelle ich sie gerade auf
onPrem um)
Agenten könnte
Möglichkeit gegeben
werden mit
zusätzlichen Systemen
zu Kommunizieren.
Externe Systeme
RAG
DB's
...

Abbildung 1

Zeitplan - Mit hilfe von KI erstellt - GPT 6.1 Sol

Nr.	Schritt	Aufgabe und konkretes Ergebnis	Ungefähre Dauer	Zeitraum	Nächstes JourFix
1	Thema und Untersuchungsumfang verbindlich abgrenzen	Leitfrage, Unterfragen, Anwendungsfall und erwartete Ergebnisse mit der Betreuung abstimmen *(Das machen wir gerade schon - Expose)* Interviewtermine anfragen und Rückfalloption festlegen. Ergebnis: abgestimmtes Untersuchungsdesign.	3 Arbeitstage	Tag 1–3 22.10.-05.11.	**05.11.**
2	Literatur gezielt vertiefen	Vorhandene Quellen prüfen und Lücken schließen, agentenspezifischen Bedrohungen und Evaluation schließen. *(Grundsätzlich fühle ich mich bei den Bedrohungen/Risiken quellen technisch recht sicher. Finde aktuell die Architektur quellenlage noch bisschen dürftig - Ideen?)*	2 Arbeitstage	Tag 2–5 22.10.-05.11.	**05.11.**
4	Fiktive Firma Entwickeln	Hier bietet sich an ein Telekommunikationsunternehmen auszudenken, da Branchenfragen durch die Interviewten Personen ausgedacht werden kann. Ich denke hier würde ich auch ne KI einsetzen wollen.	2 Arbeitstage	2-4 22.10.-05.11.	**05.11.**
5	Interview- und Auswertungskonzept entwickeln	Leitfaden aus den Forschungsfragen ableiten, Auswahl der Gesprächspartner begründen und Kategorien für die Auswertung vorbereiten. Ergebnis: einsatzbereiter Leitfaden und dokumentiertes Verfahren.	3 Arbeitstage	Tag 8–10 22.10.-05.11.	**05.11.**
	Technische Vorbereitung - Infrastraktur	- Agenten Checken - Produktive Agenten eigenes System geben (forken) - Planner Agent reduzieren - Orchestrator vorbereiten, sodass mit der eigentlichen Implementierung gestartet werden könnte.			**19.11.**
6	Interviews durchführen und auswerten 24.11. mit abteilungsleitung	Zwei bis drei Gespräche führen, aufbereiten und kategoriengeleitet analysieren. Ergebnis: nachvollziehbare Erkenntnisse zu Aufgaben, Randbedingungen und Schutzbedarf.	4 Arbeitstage	Tag 11–14 19.11.-27.11.	**03.12.**
7	Anforderungen und Referenzszenario spezifizieren	Interview- und Literaturergebnisse in priorisierte Anforderungen übersetzen. Eingaben, Quellen, Nutzerrollen, Ergebnisse und Abnahmekriterien festlegen. Ergebnis: User-Storie, Persona und Szenariobeschreibung.	3 Arbeitstage	Tag 15–17 27.11.-30.12	**03.12.**
8	Evaluationsdesign vorab festlegen	Vergleichsbedingungen, Aufgaben, Sicherheitsfälle, Bewertungsraster und Wiederholungen definieren. Ergebnis: Evaluationsprotokoll, bevor Implementierung und Ergebnisse die Kriterien beeinflussen.	4 Arbeitstage	Tag 18–21 30.11.-03.12.	**03.12.**
9	Ausgangsarchitekturen entwerfen	Multi-Agenten-Entwurf mit Rollen, Schnittstellen, Datenflüssen und Annahmen beschreiben. Ergebnis: Architekturmodell und erste Entscheidungsdokumentation.	4 Arbeitstage	Tag 22–25 03.12.-08.12.	**17.12.**
10	Strukturiertes Threat Modeling durchführen	Schutzgüter, Angreifer, Einstiegspunkte und Vertrauensgrenzen bestimmen; klassische und LLM-spezifische Bedrohungen erheben und priorisieren. Ergebnis: Bedrohungsregister mit begründeter Bewertung.	5 Arbeitstage	Tag 26–30 08.12.-11.12.	**17.12.**
11	Gegenmaßnahmen ableiten und Architektur überarbeiten	Priorisierte Bedrohungen in technische und organisatorische Kontrollen übersetzen. Änderungen gegenüber dem Ausgangsentwurf dokumentieren. Ergebnis: sicherheitsorientierte Zielarchitektur einschließlich Restrisiken.	4 Arbeitstage	Tag 31–34 14.12.-17.12.	**17.12.**
12	Minimalen Prototyp und Baseline umsetzen	Den Referenzprozess mit wenigen Agenten, einer begrenzten Werkzeug- und Datenanbindung, ausgewählten Kontrollen und Testprotokollierung realisieren. Ergebnis: ausführbares, Versuchssystem	6 Arbeitstage	Tag 35–40 18.12.-04.01.	Mo. ??? 4.01.
13	Funktionale Vergleichsevaluation durchführen	Beide Varianten mit denselben Aufgaben und Bewertungskriterien prüfen. Ergebnisqualität, Quellenbezug, Laufzeit sowie Modell- und Tool-Aufrufe erfassen. Ergebnis: dokumentierter Vergleichsdatensatz.	4 Arbeitstage	Tag 41–44 18.12.-04.01.	Mo. ??? 4.01.
14	Sicherheits- und Fehlerszenarien prüfen	Ausgewählte Bedrohungen und Kontrollmechanismen testen, etwa manipulierte Quellen, unzulässige Werkzeuganfragen oder fehlerhafte Delegation. Ergebnis: Testprotokolle und Bewertung verbleibender Grenzen.	4 Arbeitstage	Tag 45–48 04.01.-07.01	**Do. 14.01.**
15	Ergebnisse auswerten und Forschungsfragen beantworten	Anforderungen, Entscheidungen, Bedrohungen und Testergebnisse zusammenführen. Mehrwert, Zusatzaufwand und Grenzen des MAS bewerten. Ergebnis: Evaluation, Diskussion und begründete Antworten auf die Forschungsfragen.	4 Arbeitstage	Tag 49–52 07.01.-12.01.	**Do. 28.01.**
16	Gesamtdokument konsolidieren	Bereits geschriebene Kapitel verbinden, Argumentation und Terminologie vereinheitlichen sowie Einleitung, Fazit, Abbildungen, Literatur und Anhänge vervollständigen. Ergebnis: vollständige abgabenahe Fassung.	3 Arbeitstage	Tag 53–55 12.01.-28.01.	**Do. 28.01.**
17	Qualitätssicherung, Puffer und Abgabe	Quellen und Verweise prüfen, Reproduzierbarkeit kontrollieren, Korrekturen und Layout abschließen sowie PDF und Abgabeunterlagen überprüfen. Enthält etwa zwei Tage für kleinere Verzögerungen.	5 Arbeitstage	Tag 56–60 28.01.-11.02.	**Do. 11.02.**
	ABGABE: 26.02.	Der Plan sieht vor das teile der Kapitel während dieser Schritte entstehen.	60 Arbeitstage plus/minus	ca. 3 Monate	

Literatur

[1] *open-webui/open-webui*. – URL https://github.com/open-webui/open-webui. – Zugriffsdatum: 2026-07-02. – original-date: 2023-10-06T22:08:27Z

[2] : *Produktionsreife Container-Orchestrierung*. – URL https://kubernetes.io/de/. – Zugriffsdatum: 2026-07-07

[3] : *Threat Modelin Process (Historical) - OWASP*. – URL https://owasp.org/www-community/Threat_Modeling_Process

[4] : *Welcome to Python.org*. – URL https://www.python.org/. – Zugriffsdatum: 2026-07-02

[5] : *Welcome to the Foundation for Intelligent Physical Agents*. – URL https://www.fipa.org/. – Zugriffsdatum: 2026-07-03

[6] Anbiaee, Zeynab ; Rabbani, Mahdi ; Mirani, Mansur ; Piya, Gunjan ; Opushnyev, Igor ; Ghorbani, Ali ; Dadkhah, Sajjad: *Security Threat Modeling for Emerging AI-Agent Protocols: A Comparative Analysis of MCP, A2A, Agora, and ANP*. – URL https://arxiv.org/abs/2602.11327. – Zugriffsdatum: 2026-07-01. – Version Number: 2

[7] Andric, Sandro: *(PDF) LLM Hijacking: When Models Manipulate Their Routers*. – URL https://www.researchgate.net/publication/398334488_LLM_Hijacking_When_Models_Manipulate_Their_Routers. – Zugriffsdatum: 2026-07-01

[8] Botti, V.: *Agentic AI and Multiagentic: Are We Reinventing the Wheel?*. – URL https://arxiv.org/abs/2506.01463. – Zugriffsdatum: 2026-06-17. – Version Number: 1

[9] Chalupsky, Hans ; Finin, Tim ; Fritzson, Rich ; Mckay, Don ; Shapiro, Stuart ; Wiederhold, Gio: An Overview of KQML: A Knowledge Query and Manipulation Language.

[10] Chase, Harrison: *LangChain*. – URL https://github.com/langchain-ai/langchain. – Zugriffsdatum: 2026-07-02. – original-date: 2022-10-17T02:58:36Z

[11] Editor, OWASPGenAIProject: *OWASP Top 10 for LLM Applications 2025*. – URL https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/. – Zugriffsdatum: 2026-07-03

[12] Ferrag, Mohamed A. ; Tihanyi, Norbert ; Hamouda, Djallel ; Maglaras, Leandros ; Lakas, Abderrahmane ; Debbah, Merouane: From Prompt Injections to Protocol Exploits: Threats in LLM-Powered AI Agents Workflows. 12, Nr. 2, S. 353–383. – URL http://arxiv.org/abs/2506.23260. – Zugriffsdatum: 2026-06-17. – ISSN 24059595

[13] Georgeff, M. P.: Communication and interaction in multi-agent planning.

[14] Jennings, Nicholas R. ; Sycara, Katia ; Wooldridge, Michael: A Roadmap of Agent Research and Development. 1, Nr. 1, S. 7–38. – URL https://link.springer.com/10.1023/A:1010090405266. – Zugriffsdatum: 2026-06-19. – ISSN 1387-2532, 1573-7454

[15] Jennings, Nick R.: Commitments and conventions: The foundation of coordination in multi-agent systems. 8, Nr. 3, S. 223–250. – URL https://www.cambridge.org/core/product/identifier/S0269888900000205/type/journal_article. – Zugriffsdatum: 2026-06-30. – ISSN 0269-8889, 1469-8005

[16] Khan, Muhammad S. ; Khan, Muhammad J. ; Sharif, Muhammad ; Yasmin, Sadaf: Beyond Single-Framework Architectures: A Systematic Evaluation and Hybrid Design for Scalable Multi-Agent Coordination. 14, S. 59688–59705. – URL https://ieeexplore.ieee.org/document/11481053/. – Zugriffsdatum: 2026-05-22. – ISSN 2169-3536

[17] Krawiecka, Klaudia ; Witt, Christian S. de: *Extending the OWASP Multi-Agentic System Threat Modeling Guide: Insights from Multi-Agent Security Research*. – URL https://arxiv.org/abs/2508.09815. – Zugriffsdatum: 2026-07-01. – Version Number: 1

[18] Lesser, V.R.: A retrospective view of FA/C distributed problem solving. 21, Nr. 6, S. 1347–1362. – URL http://ieeexplore.ieee.org/document/135681/. – Zugriffsdatum: 2026-06-22. – ISSN 00189472

[19] OWASP: *OWASP Threat Modeling Project*. – URL https://owasp.org/www-project-threat-modeling/

[20] Ramírez, Sebastián: *FastAPI*. – URL https://github.com/fastapi/fastapi. – Zugriffsdatum: 2026-07-02. – original-date: 2018-12-08T08:21:47Z

[21] Raza, Shaina ; Sapkota, Ranjan ; Karkee, Manoj ; Emmanouilidis, Christos: TRiSM for Agentic AI: A review of Trust, Risk, and Security Management in LLM-based Agentic Multi-Agent Systems. 7, S. 71–95. – URL https://linkinghub.elsevier.com/retrieve/pii/S2666651026000069. – Zugriffsdatum: 2026-06-30. – ISSN 26666510

[22] Russell, Stuart J. ; Norvig, Peter: *Artificial intelligence: a modern approach*. Fourth edition, global edition. Pearson (Prentice Hall series in artificial intelligence). – ISBN 978-1-292-40113-3 978-1-292-40117-1

[23] Shostack, Adam: *Threat modeling: Designing for security*. John wiley & sons, 2014

[24] Sycara, K. ; Pannu, A. ; Willamson, M. ; Dajun Zeng ; Decker, K.: Distributed intelligent agents. 11, Nr. 6, S. 36–46. – URL http://ieeexplore.ieee.org/document/546581/. – Zugriffsdatum: 2026-07-01. – ISSN 0885-9000

[25] Tete, Stephen B.: *Threat Modelling and Risk Analysis for Large Language Model (LLM)-Powered Applications*. – URL http://arxiv.org/abs/2406.11007. – Zugriffsdatum: 2026-06-17

[26] Vashishtha, Vansh ; Pandey, Sonal ; Singh, Yash R. ; Bahga, Arshdeep ; Madisetti, Vijay K.: COSMIC: Leader-Driven Context-Oriented Collaboration Between Agents. 13, Nr. 3, S. 3262–3287. – URL https://ieeexplore.ieee.org/document/11417193/. – Zugriffsdatum: 2026-06-30. – ISSN 2329-924X, 2373-7476

[27] Wang, Yuntao ; Pan, Yanghe ; Su, Zhou ; Deng, Yi ; Zhao, Quan ; Du, Linkang ; Luan, Tom H. ; Kang, Jiawen ; Niyato, Dusit: Large Model-Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends. 28, S. 1906–1949. – URL https://ieeexplore.ieee.org/document/11022699/. – Zugriffsdatum: 2026-06-30. – ISSN 1553-877X, 2373-745X

[28] Wong, H. C. ; Sycara, Katia: Adding security and trust to multiagent systems. 14, Nr. 9, S. 927–941. – URL http://www.tandfonline.com/doi/abs/10.1080/08839510050144886. – Zugriffsdatum: 2026-07-01. – ISSN 0883-9514, 1087-6545

[29] Wooldridge, Michael ; Jennings, Nicholas R.: Intelligent agents: theory and practice. 10, Nr. 2, S. 115–152. – URL https://www.cambridge.org/core/product/identifier/S0269888900008122/type/journal_article. – Zugriffsdatum: 2026-06-22. – ISSN 0269-8889, 1469-8005

[30] Wooldridge, Michael J.: *An introduction to multiagent systems*. J. Wiley. – ISBN 978-0-471-49691-5

[31] Xue, Liwei ; Liu, Guo-Ping ; Hu, Wenshan: All-in-One Framework for Design, Simulation, and Practical Implementation of Distributed Multiagent Control Systems. 54, Nr. 7, S. 4331–4344. – URL https://ieeexplore.ieee.org/document/10493161/. – Zugriffsdatum: 2026-06-30. – ISSN 2168-2216, 2168-2232

[32] Yan, Bingyu ; Zhou, Zhibo ; Zhang, Litian ; Zhang, Lian ; Zhou, Ziyi ; Miao, Dezhuang ; Li, Zhoujun ; Li, Chaozhuo ; Zhang, Xiaoming: *Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems*. – URL https://arxiv.org/abs/2502.14321. – Zugriffsdatum: 2026-06-30. – Version Number: 3

[33] Zhang, Haiyue ; Nian, Yi ; Zhao, Yue: *Agent Audit: A Security Analysis System for LLM Agent Applications*. – URL https://arxiv.org/abs/2603.22853. – Zugriffsdatum: 2026-07-01. – Version Number: 1